

Εργασία 2025-2026

Ασφάλεια Πληροφοριακών Συστημάτων

Βασίλειος Κουδούνης
10739

koudounis@auth.gr

1 Εισαγωγή

- Σκοπός αυτής της εργασίας είναι ο εντοπισμός και η τεκμηρίωση των κενών ασφαλείας μέσω τεχνικών ελέγχου διείδυσης (Penetration Testing).
- Ο σχεδιασμός διορθωτικών ενεργειών στον πηγαίο κώδικα (Remediation).
- Η τελική επαλήθευση της ασφάλειας του συστήματος.

2 Security Problem Detection

Εδώ παρουσιάζονται τα προβλήματα ασφαλείας που εντοπίστηκαν μαζί με τον ευπαθή κώδικα.

2.1 SQL Injection - Παράκαμψη Αυθεντικοποίησης

Στην φόρμα εισόδου (login.php) διαπιστώνεται ότι η εφαρμογή συνενώνει απευθείας τις εισόδους του χρήστη χωρίς έλεγχο, κατασκευάζοντας έτσι το ερώτημα SQL με τρόπο ώστε ο χρήστης να μπορεί να παρέμβει.

- Αρχείο κώδικα: login.php
- Απόσπασμα κώδικα:

```
$sql_query = "SELECT * FROM login_users WHERE username='{$username}'  
AND password='{$password}';";
```

- Τρόπος επίθεσης: Στο πεδίο username εισάγουμε τη συμβολοσειρά:

```
' OR 1=1 #
```

και το σύστημα ταυτοποίησε τον εισβολέα ως τον πρώτο χρήστη της βάσης (Administrator) χωρίς να απαιτηθεί καν κωδικός πρόσβασης.

Entries of ' OR 1=1

www.test.com	
Username:	Password:
tom	tompass
Delete	

website
Username
Password
Insert new website

[Notes - announcements](#)

[Logout](#)

[Home page](#)

2.2 Stored Cross-Site Scripting (XSS)

Σε αυτή την περίπτωση, ο κακόβουλος κώδικας 'αποθηκεύεται' μόνιμα στη βάση δεδομένων της εφαρμογής και εκτελείται στον browser οποιουδήποτε χρήστη επισκεφθεί τη σελίδα. Στην εφαρμογή μας, το πρόβλημα εντοπίζεται στη σελίδα των ανακοινώσεων/σημειώσεων (notes.php), όπου τα δεδομένα εισάγονται χωρίς έλεγχο.

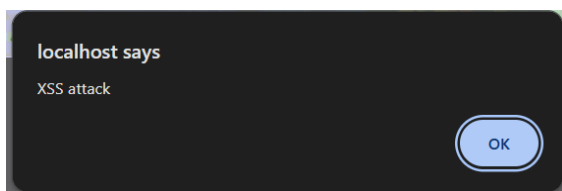
- Απόσπασμα κώδικα:

```
echo "<div class='note-content'>" . $row["note"] . "</div>";
```

- Σενάριο Επίθεσης: Εισαγωγή κακόβουλου script:

```
<script>alert('XSS Attack');</script>
```

,το οποίο έχει ως αποτέλεσμα την εμφάνιση αναδυόμενου παραθύρου (alert box) σε όλους τους χρήστες, αναδεικνύοντας τη δυνατότητα εκτέλεσης κώδικα.



#	1 id	2 username	3 password
1	1	u1	p1

Σχήμα 1: Αποθήκευση κωδικών σε απλή μορφή

2.3 Μη ασφαλής Αποθήκευση Κωδικών (Plaintext Storage)

Οι κωδικοί πρόσβασης αποθηκεύονται ως απλό κείμενο στην βάση δεδομένων.

- Αρχείο κώδικα: register.php
- Απόσπασμα κώδικα:

```
// Insert a new user
$sql_query = "INSERT INTO login_users (username,password) VALUES ('{$new_username}', '{$new_password}');
```

Αυτό έχει ως αποτέλεσμα την έκθεση όλων των κωδικών των χρηστών σε μη-εξειδικευμένο χρήστη που μπορεί να αποκτήσει πρόσβαση στην βάση δεδομένων.

2.4 Παραβίαση Αρχής Ελαχίστων Προνομίων

Η Παραβίαση της Αρχής των Ελαχίστων Προνομίων (Principle of Least Privilege) επιβεβαιώνεται από το γεγονός ότι η εφαρμογή συνδέεται στη βάση δεδομένων με πλήρη δικαιώματα διαχειριστή, αντί για έναν περιορισμένο χρήστη.

Ο χρήστης "root" είναι ο προεπιλεγμένος υπερχρήστης της MySQL που έχει δικαίωμα να κάνει τα πάντα (διαγραφή βάσεων, αλλαγή χρηστών κ.λπ.). Οποιαδήποτε επιτυχής επίθεση SQL Injection θα έδινε στον επιθέμενο πλήρη έλεγχο ολόκληρου του διακομιστή βάσης δεδομένων.

3 Προτεινόμενες Λύσεις Και Υλοποίηση

Παραθέτονται οι δομικές και λογικές αλλαγές που πραγματοποιήθηκαν.

3.1 Χρήση Prepared Statements

Η χρήση Prepared Statements είναι η πιο αποτελεσματική μέθοδος για την εξάλειψη του SQL Injection, καθώς διαχωρίζει πλήρως την εντολή SQL από τα δεδομένα που εισάγει ο χρήστης.

Σε όλα τα σημεία αλληλεπίδρασης με τη βάση δεδομένων (login.php, register.php, dashboard.php, notes.php), αντικαταστάθηκε η απλή εκτέλεση ερωτημάτων με Prepared Statements. Έτσι διαχωρίζονται πλήρως τα δεδομένα από τις εντολές SQL, εξαλείφοντας τον κίνδυνο SQL Injection.

Ακολουθεί ένα παράδειγμα υλοποίησης (login.php)

```
// 1. Use prepared statements
$stmt = $conn->prepare("SELECT id, username, password FROM login_users WHERE username = ?");

// 2. Join of the parameters (s = string)
```

```

$stmt->bind_param("s", $username);

// 3. Execution
$stmt->execute();

// Get the result
$result = $stmt->get_result();

```

3.2 Output Encoding

Για την πρόληψη επιθέσεων XSS, εφαρμόστηκε η συνάρτηση `htmlspecialchars()` πριν την εμφάνιση δεδομένων χρήστη στην οθόνη. Αυτή η συνάρτηση μετατρέπει το `<script>` σε `lt;scriptgt;`. Ο χρήστης θα το βλέπει ως κείμενο, αλλά ο browser δεν θα το τρέχει.

Παράδειγμα υλοποίησης (notes.php):

```

// instead of this:
// echo "<div class='note-content'>" . $row["note"] . "</div>";

// we use this (htmlspecialchars):
echo "<div class='note-content'>" . htmlspecialchars($row["note"], ENT_QUOTES, 'UTF-8')

```

3.3 Hashing

Εφαρμόζουμε την συνάρτηση `password_hash()` κατά την εγγραφή και πλέον στην βάση αποθηκεύεται μόνο το hash του κωδικού και όχι ο ίδιος ο κωδικός.

Στο register.php:

```

$new_username = trim($_POST['new_username']);
$plain_password = $_POST['new_password'];
$hashed_password = password_hash($plain_password, PASSWORD_DEFAULT);
$stmt->bind_param("ss", $new_username, $hashed_password);

```

3.4 Δημιουργία ειδικού χρήστη για την εφαρμογή

Αντί για τον root, δημιουργούμε έναν νέο χρήστη στην MySQL που θα έχει πρόσβαση μόνο στη συγκεκριμένη βάση δεδομένων και μόνο για τις απαραίτητες ενέργειες. Συγκεκριμένα θα έχει δικαιώματα SELECT, INSERT, UPDATE, DELETE.

Δημιουργήθηκε ένα νέο αρχείο config.php για την αποφυγή επαλαμβανόμενου κώδικα.

Αρχείο κώδικα config.php:

```

<?php
// config.php

```

```
// SECURITY FIX: Centralized connection logic.
// We should use a limited database user (e.g., 'app_user') instead of 'root'.
$host = "localhost";
$user = "root"; // Change this to 'app_user' if you created one in HeidiSQL
$pass = "";      // Change this to the password you set for 'app_user'
$db    = "pwd_mgr";

// Create connection
$conn = new mysqli($host, $user, $pass, $db);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

3.5 Total file changes

Συνολικά τα αρχεία που τροποποιήθηκαν είναι:

- (new) config.php: Περιέχει τις ρυθμίσεις σύνδεσης με την βάση δεδομένων.
- login.php: Έλεγχος hash κωδικών και Prepared Statements.
- register.php: Προσθήκη hashing κωδικών και prepared statements
- dashboard.php: Εφαρμογή prepared statements για εισαγωγή/διαγραφή ιστοσελίδων και html-specialchars στην προβολή.
- notes.php: Prepared Statements στην εισαγωγή και χρήση htmlspecialchars στην προβολή για αποτροπή XSS

4 Final testing

Αφού κάναμε τις διορθώσεις ελέγχουμε ξανά τα σενάρια επίθεσης για την επιβεβαίωση της ασφάλειας.

4.1 Έλεγχος του SQL Injection

Πλέον η εισαγωγή του ' OR 1=1 στο πεδίο login αποτυγχάνει, καθώς το σύστημα αναζητάει τον χρήστη με αυτό ακριβώς το όνομα που προφανώς δε βρίσκει. Δεν καταφέρνει ο επιτιθέμενος να πειράξει τον κώδικα.

Password Manager

Login

Invalid username or password

[Register new user](#)

[Home page](#)

4.2 Έλεγχος του XSS

List of notes/comments

test1

by u1

hey

by u1

<script>alert('XSS attack');</script>

by u1

<script>('XSS Attack!')</script>

by u1

<script>alert(1);</script>

by u2

Enter your note:

Write your note here...

Καταχωρήθηκε επιτυχώς νέα σημείωση με περιεχόμενο: `<script>alert(1)</script>` και ο κώδικας εμφανίστηκε ως απλό κείμενο στην οθόνη και δεν εκτελέστηκε (Sanitization).

4.3 Έλεγχος βάσης δεδομένων

Δημιουργήθηκε νέος χρήστης u2 και έλεγχος μέσω HeidiSQL. Πλέον ο κωδικός εμφανίζεται κρυπτογραφημένος.

#	1 id 🔑 ⚙	2 username 🔑 ⚙	3 password 🔑 ⚙
1	7	u2	\$2y\$10\$2M7eagVQ2SjRdbm6IOIe6OQIMT1yvsg...
2	1	u1	p1

4.4 Μη ασφαλές πρωτόκολλο HTTP

Η εφαρμογή τρέχει σε τοπικό περιβάλλον HTTP (localhost). Σε πραγματικές συνθήκες, η χρήση TLS/SSL είναι απαραίτητη για την κρυπτογράφηση του δικτύου.